

HTTP 완벽 가이드 · Week 1

HTTP 개요와 웹의 구성요소

1.1 ~ 1.10 · p.3 ~ 25

발표 1-1 · 1-2 · 정우인

PART 1-1

HTTP와 리소스, 트랜잭션

1.1 ~ 1.5 · p.3 ~ 12

URI — 리소스를 가리키는 이름

URI Uniform Resource Identifier

정보 리소스를 고유하게 식별하고, 위치를 지정할 수 있는 이름

URL Uniform Resource Locator

리소스가 어디에 있고
어떻게 접근하는지 서술

<http://www.joes-hardware.com/specials/saw-blade.gif>

URN Uniform Resource Name

위치와 무관하게
리소스에 붙인 고유한 이름

urn:ietf:rfc:2141

오늘날 HTTP에서는 **URI ≈ URL** — URN은 아직 실험 단계라 거의 쓰이지 않는다

URL은 세 부분으로 읽는다

http://

www.joes-hardware.com

/specials/saw-blade.gif

1

2

3

1 스킴

어떤 프로토콜로 접근할지
(여기서는 HTTP)

2 서버 주소

리소스를 가진
서버의 인터넷 주소

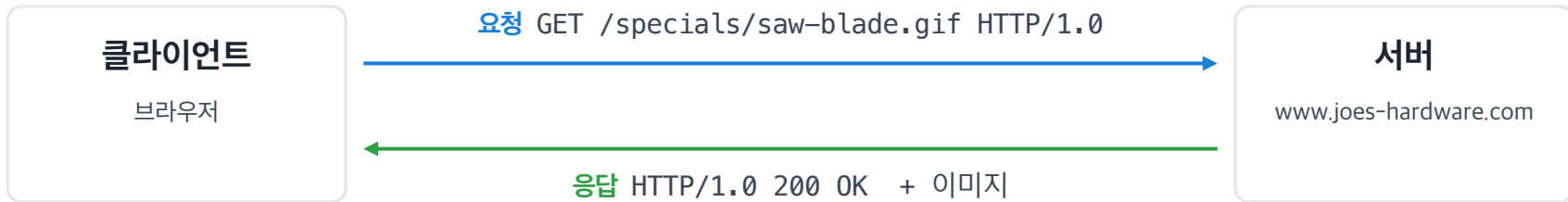
3 리소스 경로

그 웹 서버 안에서
리소스가 있는 위치

포트, 파라미터, 질의 문자열 등 URL의 세부 문법은 2장(발표 1-3)에서 자세히 다룬다

1.4 트랜잭션

HTTP 트랜잭션 = 요청 하나 + 응답 하나



일반적인 트랜잭션 (DB 등)

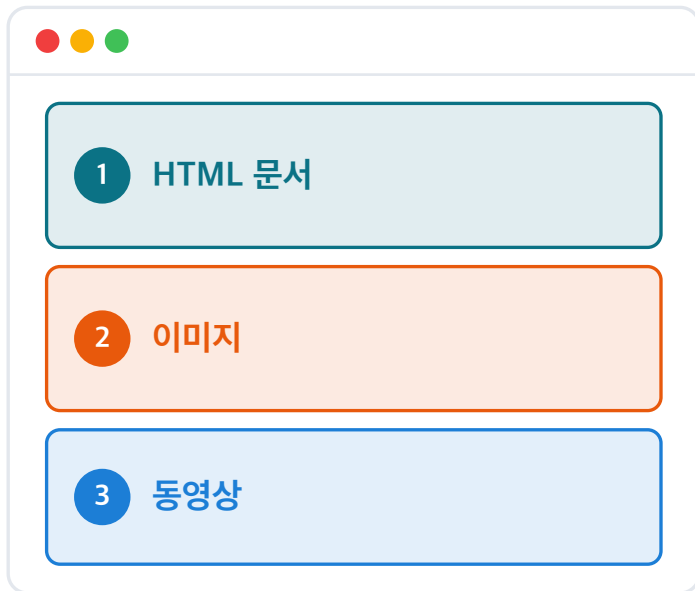
여러 작업을 하나로 묶어
전부 성공하거나, 전부 취소하는 단위

HTTP 트랜잭션

요청 메시지 하나와 응답 메시지 하나의 쌍
— 주고받기 한 번이 곧 한 단위

메서드(GET 등)와 상태 코드(200 등)는 3장(Week 2)에서 자세히 다룬다

웹 페이지 하나 = 여러 개의 트랜잭션



1 HTML 요청 / 응답
www.joes-hardware.com

2 이미지 요청 / 응답
www.joes-hardware.com

3 동영상 요청 / 응답
다른 서버일 수도 있다 (예: 영상 전용 서버)

웹 페이지는 리소스 하나가 아니라 **리소스의 모음**이고,
그 리소스들은 서로 다른 서버에 있을 수도 있다

웹 서버와 WAS는 무엇이 다른가

Web Server Nginx · Apache

정적 파일 서빙

HTTP 요청 처리

SSL 처리

로드밸런싱

✗ Java 코드를 직접 실행하지는 않는다

WAS Tomcat

Web Application Server

웹 서버 기능

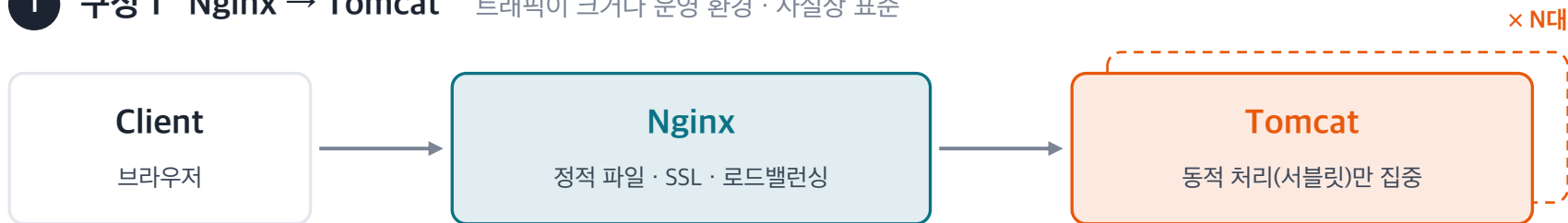
+

Servlet 실행 환경

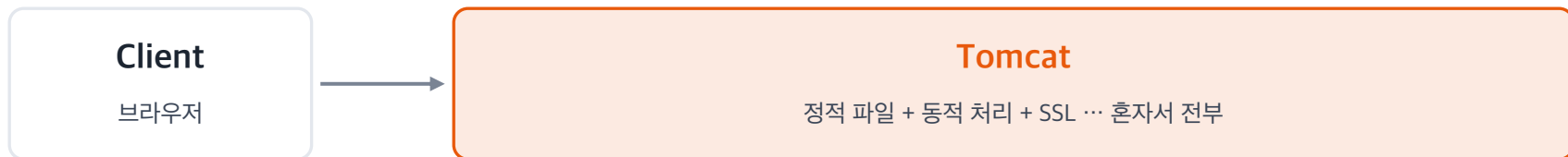
✓ 실제 서블릿(Java) 코드를 실행한다

실무에서 보는 두 가지 구성

1 구성 1 Nginx → Tomcat 트래픽이 크거나 운영 환경 · 사실상 표준



2 구성 2 Tomcat 단독 소규모 · 개인 프로젝트



왜 Tomcat 앞에 Nginx를 둘까?

1 정적 파일

이미지·CSS·JS 응답은
Nginx가 훨씬 빠르고 가볍다

2 로드밸런싱

여러 대로 늘린 Tomcat에
요청을 나눠 보낸다

3 SSL 종료

인증서·암복호화는 Nginx가,
뒤의 Tomcat은 순수 HTTP

4 보안

내부 서버를 숨기고
비정상 트래픽을 1차로 거른다

5 라우팅

여러 도메인·백엔드를
한 곳에서 관리한다

각자 잘하는 일만

Nginx 앞단 처리 전문
Tomcat 동적 로직에 집중

PART 1-2

TCP 커넥션과 웹의 구성요소

1.6 ~ 1.10 · p.13 ~ 25

HTTP는 TCP/IP 위에서 동작한다

HTTP

애플리케이션 계층

TCP

전송 계층

IP

네트워크 계층

TCP = 신뢰성 있는 의사소통

클라이언트와 서버가 주고받는 메시지가

✓ 없어지지 않고

✓ 손상되지 않고

✓ 순서가 뒤바뀌지 않고

그대로 도착한다

HTTP를 하기 전에 필요한 것

1

HTTP 메시지를 보내려면
먼저 TCP/IP 커넥션을
맺어야 한다

2

TCP 커넥션을 맺으려면
IP 주소와 포트 번호가
필요하다

3

브라우저는 URL만 아는데,
IP 주소와 포트 번호는
어떻게 알아낼까?

필요한 두 가지 **IP 주소** + **포트 번호**

URL에서 IP 주소와 포트 찾기

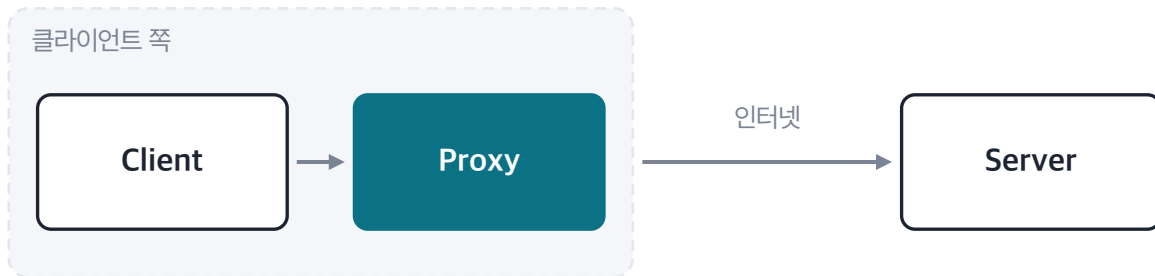


→ 알아낸 IP 주소의 443번 포트로 TCP 커넥션을 맺고, 그 위에서 HTTP 메시지를 주고받는다

포워드 프락시 vs 리버스 프락시

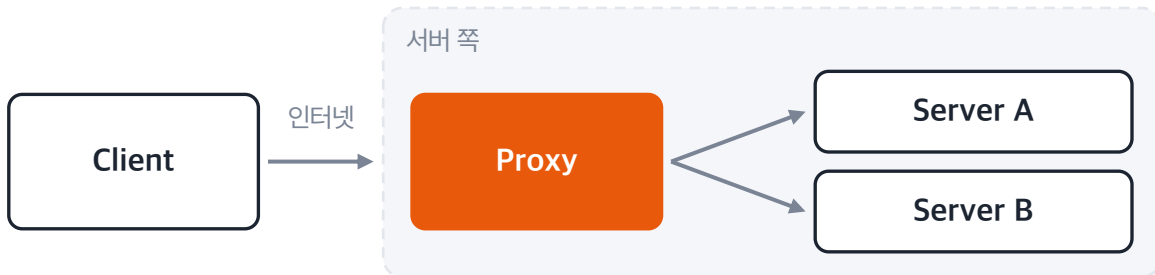
포워드 프락시

클라이언트 쪽에서
“내가 대신 요청해줄게”
예: VPN



리버스 프락시

서버 앞단에 서서
“요청을 받아 내부 서버로
전달해줄게” 예: Nginx



리버스 프락시(Nginx)가 하는 일

1

로드밸런싱

백엔드 인스턴스가 여러 개일 때 요청을 나눠 분산

2

SSL 종료

HTTPS 암호화하는 Nginx가, 내부로는 평문 HTTP

3

캐싱

자주 오는 응답을 저장해 두고 백엔드 없이 바로 응답

4

요청 필터링 · 보안

악성·과도한 요청을 백엔드 앞에서 걸러냄

5

라우팅

경로·도메인에 따라 다른 백엔드로

/api/users



user 서비스

/api/orders



order 서비스

MSA에서도 클라이언트는
Nginx 하나만 바라보면 된다

애플리케이션 캐시 — Spring + Redis

```
@Cacheable(value = "products", key = "#id")  
public Product findProduct(Long id) {  
    return productRepository.findById(id);  
}
```

Spring이 Redis를 먼저 확인한다

cache hit

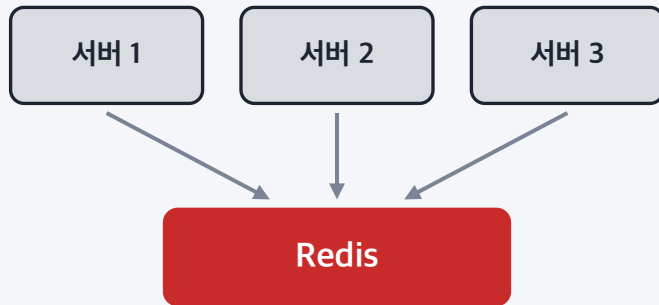
값이 있으면 → 그대로 반환

cache miss

값이 없으면 → 메서드 실행 후 저장

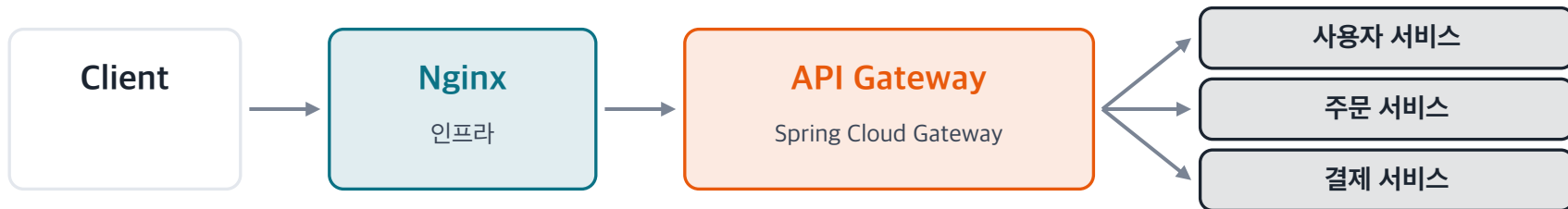
그냥 서버 메모리에 캐싱하면?

서버가 여러 대면 서버마다 캐시가 달라진다



별도 인프라 → 모든 서버가 같은 캐시를 본다

API 게이트웨이 — 서비스 앞의 문지기



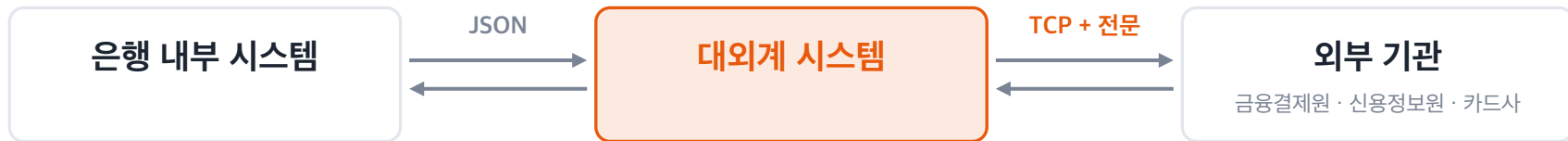
Nginx 라우팅 인프라 레벨

설정으로 경로·도메인을 나눠 전달

Spring Cloud Gateway 애플리케이션 레벨

Java 코드로 라우팅·인증 같은
로직을 직접 짜 넣을 수 있다

금융권 대외계 — 프로토콜을 갈아 끼우는 게이트웨이



전문 정해진 자리수에 값을 채워 넣은 하나의 긴 문자열 (REST API가 아님)

